

Week 07

Functional Coverage and Metrics

Jesus Sanchez

Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

Processor Verification

Master in Innovation and Research in Informatics

Objectives of the Week

- Understand **functional coverage** and its importance in verification
- Learn how to create/integrate **covergroups and coverpoints**
- **Analyze coverage results** and use them to improve verification quality

Agenda

1. Introduction to functional coverage
2. Advanced coverage features
3. Examples and exercises
4. Wrap-up and self-learning

What is Functional Coverage?

■ Definition

- A metric that measures how much of the design's intended behavior has been exercised
- Complements code coverage (e.g., line, toggle) by focusing on what the design is supposed to do

■ Key Concept

- Functional coverage is defined by the verification engineer, not inferred from the code

■ Examples

- Did the ALU receive all operation types (add, sub, mul)?
- Were edge cases (zero, max value, overflow) tested?

■ Used With

- Covergroups, coverpoints, and bins in SystemVerilog
- Randomized and directed testbenches

Goal: Ensure all functional scenarios have been tested

Functional Coverage vs Code Coverage

■ Code Coverage

- Measures which parts of the code were executed
- Types:
 - Line coverage
 - Toggle coverage
 - Branch/condition coverage
- Generated **automatically** by simulation tools



■ Functional Coverage

- Measures which scenarios or behaviors were tested
- Focused on **intent** rather than structure
- Defined **manually** using covergroups

Use Both Together

High code coverage $\neq$ full verification
Functional coverage checks what really matters

Covergroups and Coverpoints: Basic Concepts

- **Covergroup:** A construct that defines what functional behavior to observe
 - Declared in SystemVerilog using **covergroup ... endgroup**
- **Coverpoint:** A specific variable or expression being monitored
 - Defines the range of values or transitions to track
- **Simple Example:** 
- **How it Works**
 - Simulation records values hitting the coverpoints
 - Each **bin** corresponds to a tracked value or range
- **Use Case**
 - Track if all instruction types or corner cases were triggered during verification

```
covergroup cg;  
  coverpoint length {  
    bins short = {[0:63]};  
    bins med   = {[64:511]};  
    bins long  = {[512:1023]};  
  }  
endgroup
```

Think of covergroups as the "what to observe" and coverpoints as the "where to look"

Bins and Cross Coverage

■ Bins

- Bins define value ranges or individual values to track within a coverpoint
- Can be:
 - Implicit (auto-created)
 - Explicit (user-defined): `coverpoint opcode { bins arithmetic = {0, 1, 2}; // ADD, SUB, MUL}`

■ Cross Coverage

- Measures combinations of multiple coverpoints
- Useful for checking interactions between variables
`cross opcode, alu_mode;`

■ Why It Matters

- Ensures combinations of conditions are tested
- Helps identify untested interactions

Tip: Use `ignore_bins` and `illegal_bins` to exclude or flag certain values

Creating Covergroups: Example

```
1  ✓ covergroup alu_cg @(posedge clk);
2
3      // Coverpoint for opcode
4  ✓  coverpoint opcode {
5      bins arithmetic = {0, 1, 2};      // ADD, SUB, MUL
6      bins logic_ops = {3, 4};          // AND, OR
7      ignore_bins reserved = {7};      // Reserved opcode
8  }
9
10     // Coverpoint for alu_mode
11  ✓  coverpoint alu_mode {
12     bins mode_bins[] = {[0:3]};        // Modes 0 to 3
13     illegal_bins invalid = {5};        // Invalid mode
14 }
15
16     // Cross coverage
17     cross opcode, alu_mode;
18
19 endgroup
```

```
COVERGROUP: alu_cg (from alu_tb.sv line 12)
INSTANCES: 1
SAMPLES: 1000
COVERAGE: 83%

COVERPOINT: opcode (line 14)
BINS:
    arithmetic      90 hits (values: 0,1,2)
    logic_ops       45 hits (values: 3,4)
    reserved        0 hits (ignored)
COVERAGE: 100%

COVERPOINT: alu_mode (line 20)
BINS:
    mode_bins[0]    25 hits
    mode_bins[1]    35 hits
    mode_bins[2]    30 hits
    mode_bins[3]    0 hits
    invalid         1 hit (ILLEGAL - flagged!)
COVERAGE: 75%

CROSS: opcode x alu_mode (line 26)
CROSS BINS: 15/20 hit (75%)

TOTAL COVERAGE (alu_cg): 83%
ILLEGAL BIN VIOLATIONS:
- alu_mode.invalid hit 1 time (line 24)
```

Integrating Covergroups into Testbenches

■ Where to Place Covergroups?

- Inside testbench components (e.g., monitors, scoreboards)
- Or declared globally in the testbench

Instantiation and sampling

```
// Declaration
alu_cg cg_inst = new();

// Automatic sampling (on event)
covergroup alu_cg @(posedge clk);

// Manual sampling (if needed)
cg_inst.sample();
```

Integration in monitor example

```
always @(posedge clk) begin
    opcode_mon = dut.opcode;
    alu_mode_mon = dut.alu_mode;
    cg_inst.sample(); // Trigger sampling
end
```

Sample is used when you define a covergroup without an event, i.e., no @() sampling trigger

Agenda

1. Introduction to functional coverage
2. Advanced coverage features
3. Examples and exercises
4. Wrap-up and self-learning

Transition Coverage and Sampling Events

■ What is Transition Coverage?

- Tracks how a signal **changes between values** over time
- Useful to verify state sequences, protocol flows, or edge behaviors (e.g. FSM)

■ Example:

```
coverpoint state {  
  bins idle_to_busy = (IDLE => BUSY);  
  bins any_change = ( [*] );  
  bins transition_example = (0 => 1 => 2);  
}
```

Operator	Description	Example
<code>=></code>	Ordered transition	<code>(1 => 2)</code>
<code>[*N]</code>	Exact N repetitions	<code>(3 => 4 [*2])</code> → 3→4→4
<code>[*N:M]</code>	Range of repetitions	<code>(1 => 2 [*1:3])</code>
<code>[->N]</code>	At least N repetitions	<code>(5 => 6 [->2])</code>
<code>[=N]</code>	Exactly N clock cycles in current state	<code>(2 [=3])</code> → stay 3 cycles
<code>[=N:M]</code>	Range of N to M clock cycles in a state	<code>(7 [=2:4])</code>
<code>[->N:M]</code>	Transition after N to M repeats	<code>(0 => 1 [->1:3])</code>
<code>[*]</code>	Any number of repetitions	<code>(9 => 10 [*])</code>
<code>[*0]</code>	Zero repetitions (no transition)	<code>(1 => 2 [*0])</code>
<code>[*0:\$]</code>	Zero or more repetitions	<code>(4 => 5 [*0:\$])</code>
<code>[*1:\$]</code>	One or more repetitions	<code>(6 => 7 [*1:\$])</code>
<code>[*N:\$]</code>	N or more repetitions	<code>(1 => 3 [*2:\$])</code>
<code>[+]</code> (deprecated)	Same as <code>[*1:\$]</code> , included in some tools for compatibility	<code>(A => B [+])</code>
<code>[->N:\$]</code>	At least N repetitions	<code>(X => Y [->2:\$])</code>

Coverage Options and Pragmas

- Why Use Options?

- Fine-tune what is collected
- Control reporting and merging behavior

- Example

```
covergroup cg @(posedge clk);  
    option.per_instance = 1;  
    option.comment = "ALU opcode test";  
    option.name = "ALU_CG";  
endgroup
```

Option	Description
name	Custom name for reporting
comment	Adds a note visible in coverage reports
per_instance	Enable per-instance collection
merge_instances	Merge coverage from all instances
goal	% threshold for coverage goal

- Why Use Pragmas?

- Used to control coverage **without changing code structure**
- Improve simulation performance

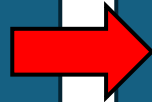
- List

Pragma	Description
// pragma coverage off	Disable coverage below this line
// pragma coverage on	Re-enable coverage
// pragma coverage block [name]	Label a block of code as a named coverage block
// pragma no_coverage	Skip coverage for line/statement

Analyzing and Reporting Coverage

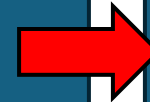
Coverage Metrics

- Code Coverage:
 - Line, Condition, Branch, Toggle
- Functional Coverage:
 - Covergroups, Coverpoints, Crosses, Bins
- Assertion Coverage (if enabled)



Coverage Report Tools

- Simulation Tools generate reports:
e.g, Mentor Questa: ucdb, vcover



Report Contents

- Summary of coverage %
- Uncovered bins and lines
- Hit counts per bin/coverpoint
- Pass/fail for coverage goals

• Common Analysis Tasks

- Identify uncovered or partially covered bins
- Refine stimulus/test scenarios
- Merge coverage across simulations
- Track coverage evolution in regressions

• Pro Tips

- Use named covergroups to organize reports
- Filter by module or instance
- Export to HTML or XML for documentation
- Prepare nice progress presentations ;-)

Example

Name	Instance Count	Coverage%	Statement%	Branch%	Toggle%	FSM State%	FSM Transition%	Condition%	Expression%	Assertion%	Directive%
Binary_to_BCD	1	81.11%	96.77%	88.23%	41.11%	100.00%	100.00%	75.00%		66.66%	
calc	1	87.18%	100.00%	89.06%	38.47%	100.00%	100.00%	57.14%	100.00%	100.00%	100.00%
display_bcd	1	64.26%	100.00%	96.15%	10.92%					50.00%	
intf	1	58.96%	75.69%	50.00%	2.43%			66.66%		100.00%	
keypad	1	88.72%	93.47%	89.47%	100.00%	100.00%	71.42%	66.66%		100.00%	
keypad_tb	1	81.23%	100.00%	100.00%	43.69%						
seven_segment	11	63.96%	61.90%	60.00%	70.00%						
stack	1	81.75%	100.00%	100.00%	27.02%			100.00%			

Assertions are typically part of the coverage report

Merging Coverage Data

■ Why Merge Coverage Data?

- Aggregate results from multiple test runs
- Improve confidence in coverage metrics
- Enable team-based or distributed verification

■ When to Merge?

- Running regression tests across configurations
- Combining coverage from constrained-random tests
- Merging targeted corner-case tests

■ How to Merge (QuestaSim Example)?

```
$ vcover merge merged.ucdb run1.ucdb run2.ucdb run3.ucdb  
$ vcover report merged.ucdb
```

Functional Coverage in Regressions

- Why Use Functional Coverage in Regressions?
 - Measure test effectiveness across large test suites
 - Identify unexercised functionality
 - Drive coverage closure over time
- Regression Flow with Coverage: test → coverage → merge → report
 1. Run different test cases and save coverage separately

```
$ vsim -c -do "run -all; coverage save test1.ucdb" work.top_tb +TEST=case1
$ vsim -c -do "run -all; coverage save test2.ucdb" work.top_tb +TEST=case2
$ vsim -c -do "run -all; coverage save test3.ucdb" work.top_tb +TEST=case3
```
 2. Merge all UCDB files:

```
$ vcover merge merged.ucdb run1.ucdb run2.ucdb run3.ucdb
```
 3. Generate report:

```
$ vcover report merged.ucdb
```

10:00

Agenda

1. Introduction to functional coverage
2. Advanced coverage features
3. Examples and exercises
4. Wrap-up and self-learning

Exercise #1

- An ALU supports operations {ADD, SUB, MUL, DIV}. Operands A and B are 32-bit signed integers.
 - Define operand categories:
 - neg : $A < 0$
 - zero : $A == 0$
 - pos : $A > 0$
- **Tasks**
 - Create a covergroup that:
 - Covers each ALU opcode.
 - Covers operand categories for A and B.
 - Crosses opcode $\times$ operand category A $\times$ operand category B.
 - Add an ignore_bins for $\text{DIV} \times B == \text{zero}$.
 - Add an illegal_bins for $\text{MUL} \times A == 0 \times B == 0$ (architecturally meaningless case)

Solution #1

```
0: typedef enum logic [1:0] {ADD, SUB, MUL, DIV} alu_op_e;
1:
2: function automatic string cat_sint32 (int signed v);
3:   if (v < 0)      return "neg";
4:   else if (v == 0) return "zero";
5:   else           return "pos";
6: endfunction
7:
8: covergroup cg_alu (alu_op_e op, int signed A, int signed B) @(posedge clk);
9:
10:  // Opcode coverage
11:  cp_opcode : coverpoint op {
12:    bins add = {ADD};
13:    bins sub = {SUB};
14:    bins mul = {MUL};
15:    bins div = {DIV};
16:  }
17:
18:  // Operand categories
19:  cp_cat_A : coverpoint cat_sint32(A) {
20:    bins neg = {"neg"};
21:    bins zero = {"zero"};
22:    bins pos = {"pos"};
23:  }
```

```
24:
25:  cp_cat_B : coverpoint cat_sint32(B) {
26:    bins neg = {"neg"};
27:    bins zero = {"zero"};
28:    bins pos = {"pos"};
29:  }
30:
31:  // Cross: opcode × operand categories
32:  x_op_A_B : cross cp_opcode, cp_cat_A, cp_cat_B {
33:
34:    // Ignore DIV × B == zero (division by zero not legal)
35:    ignore_bins div_by_zero =
36:      binsof(cp_opcode.div) && binsof(cp_cat_B.zero);
37:
38:    // Illegal: MUL × A == 0 × B == 0 (meaningless)
39:    illegal_bins mul_zero_zero =
40:      binsof(cp_opcode.mul) &&
41:      binsof(cp_cat_A.zero) &&
42:      binsof(cp_cat_B.zero);
43:  }
44:
45: endgroup
```

Instead of embedding comparisons everywhere:

```
SystemVerilog
bins neg = {[-2**31 : -1]};
bins zero = {0};
bins pos = {[1 : 2**31-1]};
```

...you get a clean, declarative coverpoint:

```
SystemVerilog
cp_cat_A : coverpoint cat_sint32(A) {
  bins neg = {"neg"};
  bins zero = {"zero"};
  bins pos = {"pos"};
}
```

Much easier for students — and for real verification engineers.

Exercise #2

- A networking IP block receives packets with the following header fields:
 - pkt_type : {DATA, CONTROL, ERROR}
 - priority : 2-bit field (0–3)
 - length : 10-bit field, but legal values are:
 - 0–63 → small
 - 64–511 → medium
 - 512–1023 → large
- Tasks
 - Write a covergroup that:
 - Creates bins for each packet type.
 - Creates named bins for the three length ranges.
 - Crosses pkt_type and length category.
 - Declares an illegal_bins for priority == 3 when pkt_type == CONTROL (this combination is architecturally forbidden)

Solution #2

```
1: typedef enum logic [1:0] {DATA, CONTROL, ERROR} pkt_type_e;
2:
3: function automatic string len_cat (int unsigned length);
4:   if (length <= 63)      return "small";
5:   else if (length <= 511) return "medium";
6:   else                  return "large"; // up to 1023
7: endfunction
8:
9: covergroup cg_pkt (pkt_type_e pkt_type,
10:                  logic [1:0] priority,
11:                  int unsigned length) @(posedge clk);
12:
13: // Packet type
14: cp_type : coverpoint pkt_type {
15:   bins data      = {DATA};
16:   bins control   = {CONTROL};
17:   bins error     = {ERROR};
18: }
19:
```

```
20: // Length categories
21: cp_len_cat : coverpoint len_cat(length) {
22:   bins small  = {"small"};
23:   bins medium = {"medium"};
24:   bins large  = {"large"};
25: }
26:
27: // Priority
28: cp_prio : coverpoint priority {
29:   bins p0 = {2'd0};
30:   bins p1 = {2'd1};
31:   bins p2 = {2'd2};
32:   bins p3 = {2'd3};
33: }
34:
35: // Cross: pkt_type x length category
36: x_type_len : cross cp_type, cp_len_cat;
37:
38: // Illegal: priority == 3 when pkt_type == CONTROL
39: illegal_bins illegal_ctrl_p3 =
40:   binsof(cp_type.control) && binsof(cp_prio.p3);
41:
42: endgroup
```

Exercise #3

- A power-management FSM has states: IDLE, ACTIVE, SLEEP, ERROR.
- Legal transitions:
 - IDLE → ACTIVE
 - ACTIVE → SLEEP
 - SLEEP → IDLE
 - Any state → ERROR
- Tasks
 - Write a covergroup that captures **state transitions** using bins and bins ... = (A ==> B) syntax.
 - Add an illegal_bins for ACTIVE → IDLE (not allowed by spec).
 - Add a cross between current state and next state, but exclude transitions to ERROR from the cross (they are already covered by transition bins).

Solution #3

```
1: typedef enum logic [1:0] {IDLE, ACTIVE, SLEEP, ERROR} pm_state_e;
2:
3: covergroup cg_pm_fsm (pm_state_e curr_state,
4:                      pm_state_e next_state) @(posedge clk);
5:
6:   cp_curr : coverpoint curr_state {
7:     bins idle   = {IDLE};
8:     bins active = {ACTIVE};
9:     bins sleep  = {SLEEP};
10:    bins error   = {ERROR};
11:  }
12:
13:   cp_next : coverpoint next_state {
14:     bins idle   = {IDLE};
15:     bins active = {ACTIVE};
16:     bins sleep  = {SLEEP};
17:     bins error  = {ERROR};
18:  }
19:
```

```
20: // Transition bins (sequence-style)
21: cp_trans : coverpoint {curr_state, next_state} {
22:   bins idle_to_active = (IDLE => ACTIVE);
23:   bins active_to_sleep = (ACTIVE => SLEEP);
24:   bins sleep_to_idle   = (SLEEP => IDLE);
25:
26:   // Any state → ERROR
27:   bins any_to_error[] = (IDLE, ACTIVE, SLEEP, ERROR => ERROR);
28:
29:   // Illegal transition: ACTIVE → IDLE
30:   illegal_bins active_to_idle_illegal = (ACTIVE => IDLE);
31: }
32:
33: // Cross current × next, excluding transitions to ERROR
34: x_curr_next : cross cp_curr, cp_next {
35:
36:   // Exclude all transitions where next_state == ERROR
37:   ignore_bins to_error =
38:     binsof(cp_next.error);
39:
40: }
41:
42: endgroup
```

Agenda

1. Introduction to functional coverage
2. Advanced coverage features
3. Examples and exercises
4. Wrap-up and self-learning

Summary of Key Concepts from this Session

■ Functional Coverage – Key Takeaways

- Tracks what functionality has been exercised in your tests
- Covergroups with coverpoints, bins, cross coverage
- Merge and analyze coverage over multiple tests or regressions

■ Big Picture

- Assertions = "Is the design behaving correctly?"
- Functional Coverage = "Have we tested all expected scenarios?"

Link to Next Week

- **Week 08: Introduction to UVM (Universal Verification Methodology)**
 - Build powerful testbenches to drive stimulus and observe DUT behavior
 - Why UVM?
 - Industry-standard SystemVerilog-based methodology
 - Promotes reuse, scalability, and constrained-random testing
 - Structured testbenches using components, TLM, factories, and sequences

Self-Learning Corner

■ Books

- SystemVerilog for Verification (3rd Ed.) by Chris Spear
 - Chapter 12: Functional Coverage

■ Videos

- Functional Coverage vs Code Coverage
https://youtu.be/m76FwxdWWpY?si=zYStQ_fDVOwwirV1

■ Articles

- <https://www.vlsiverification.net/tutorials/coverage/index.html>

“If you cannot measure it, you cannot improve it.”

Lord Kelvin